Practical Work 1: TCP File Transfer

Le Quy Ton

December 15, 2025

1. Introduction

This report documents an individual implementation of a one-to-one TCP file transfer utility (client & server) using POSIX sockets in C. The purpose is to design a minimal protocol, implement the transfer over a reliable TCP stream, and demonstrate end-to-end file delivery.

2. Protocol Design

The protocol uses short ASCII headers exchanged before streaming raw binary data. It is intentionally simple to avoid parsing complexity and to make testing straightforward.

Protocol Steps

1. Client connects to server (TCP three-way handshake).
2. Client sends header: `SEND <filename>\n`.
3. Server replies: `OK`.
4. Client sends header: `SIZE <number>\n`.
5. Server replies: `OK`.
6. Client streams raw binary file data (exactly `<number>` bytes).
7. Server writes received bytes to disk under the received filename.
8. Server replies: `DONE` and the connection may close.

3. Protocol Diagram

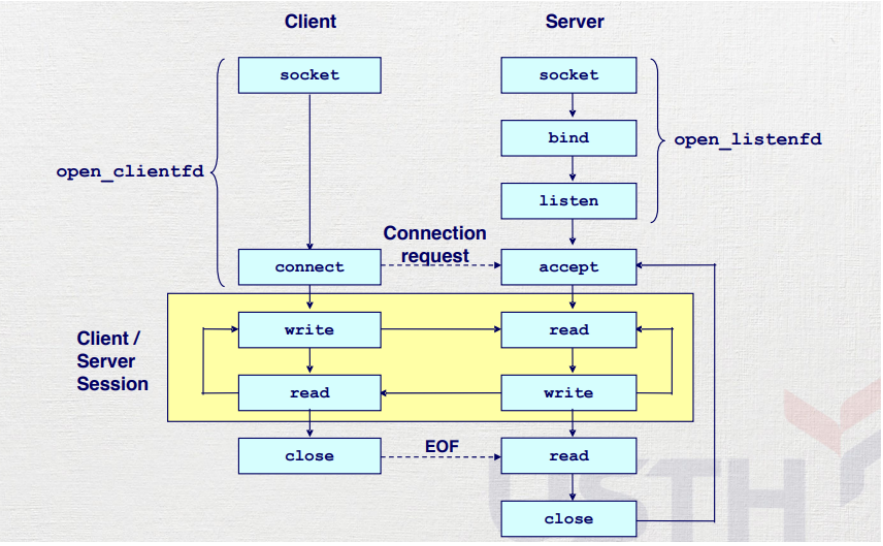


Figure 1: Socket-level workflow: open, connect/accept, write/read, close.

4. System Organization

- **Client:** Establishes TCP connection (socket(), connect()), sends metadata headers and streams file bytes (send()).
- **Server:** Listens on a port (socket(), bind(), listen()), accepts connection (accept()), reads headers, receives bytes (recv()) and writes them to disk.

5. Implementation

Below is a concise description of how the transfer is implemented and a short illustrative snippet showing the exchange of headers (not the full program).

Runtime sequence

1. Server: create socket, setsockopt(SO_REUSEADDR), bind(port), listen().
2. Client: create socket, connect(server_ip, port).
3. Client sends 'SEND <filename>\n'; server reads and responds 'OK'.
4. Client sends 'SIZE <numBytes>\n'; server responds 'OK'.

5. Client: read file in chunks and `send()` them to server.
6. Server: loop `recv()` until total bytes received == <numBytes>, write chunks to file.
7. Server sends 'DONE' and either closes or waits for next connection (design choice).

Minimal code snippet illustrating header exchange

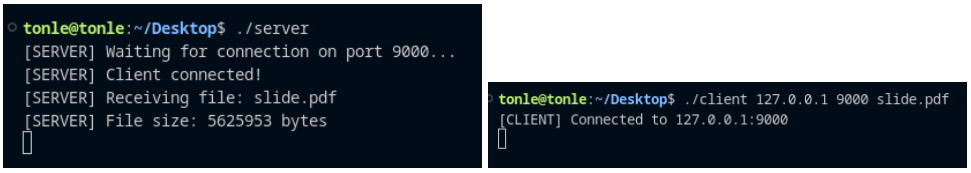
```
/* client side (concept) */
sprintf(buf,"SEND␣%s\n", filename);
send(sock, buf, strlen(buf), 0);
recv(sock, buf, sizeof(buf), 0); /* expect "OK" */

sprintf(buf,"SIZE␣%d\n", filesize);
send(sock, buf, strlen(buf), 0);
recv(sock, buf, sizeof(buf), 0); /* expect "OK" */

/* then stream binary bytes via send() */
```

Notes on robustness

- All `recv()` return values must be checked (0 = peer closed, negative = error).
- Use a loop to accumulate partial reads/writes until the declared file size is reached.
- Consider basic sanity checks: file size limit, filename validity, and handling abrupt disconnects.



7. Work distribution

This practical work was completed individually by the author. All design, implementation and testing steps were done by myself.

8. Conclusion

A minimal TCP-based file-transfer protocol was designed and implemented. The system demonstrates reliable file delivery over TCP using a small, clear protocol and can be extended for multiple clients, authentication, or reliability checks if required.